

Failure (prove the suite catches a regression) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (a real, repeatable process), **Advanced** (why one run isn't enough evidence). Read Basic first even if you already know the pattern — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

A test suite you've never actually watched fail is a suite you're just trusting blindly — you don't actually know it would catch a real problem, you're only assuming it would.

The way to check: break something on purpose. Take a working prompt, remove or change one key instruction in it, and run your test suite again. If your suite is doing its job, the score should visibly drop.

If nothing changes — if your suite still says everything passed — that's not good news. It means your suite has a hole, and you need to find and fix it before you can trust it.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

This exercise is a **test of the tests** — the same idea as intentionally breaking a smoke detector's battery to confirm the alarm actually sounds, rather than just trusting it silently sits there working.

The process: record your suite's current score as a baseline. Pick one prompt with a specific instruction in it that matters (e.g. "always mention the return policy," or "respond only in valid JSON"). Delete or weaken that one instruction. Re-run the exact same suite, unchanged. Compare the new score to the baseline.

The exact pieces:

- **Keep the original prompt safely** — copy it to a backup file, or just note the exact change so you can revert it precisely.
- **Record a real baseline number** — not "it felt fine," an actual score/pass-count from running the suite once against the unmodified prompt.
- **Change exactly one thing** — deleting one specific instruction sentence, not rewriting the whole prompt. If you change several things at once, and the score drops, you won't know *which* change your suite actually caught.
- **Re-run with zero other changes** — same tasks, same rules, same judge prompt.
- **If the score doesn't drop:** that test/rule combination has a real gap — go tighten it.

Before you run anything, write down your *prediction*: which specific test(s) do you expect to fail, and why?

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

There's a trap hiding in "run once before, run once after, compare the two numbers": if any part of your suite touches an LLM (the pipeline itself, or a judge), **a single run's score carries some noise that has nothing to do with the prompt change at all** — the model's own run-to-run variance. If your baseline score is, say, 8/10 one time and 7/10 the next *with the exact same unmodified prompt*, a genuine one-point drop after sabotage could just be noise, not proof your suite works.

The real design question: how do you tell "the score dropped because the suite caught a real regression" apart from "the score dropped because of normal run-to-run wobble"? One run of each can't answer that on its own.

The extra pieces needed:

- **Run the baseline multiple times before touching anything**, and look at its own spread (e.g. 3 runs, scores 8/10, 8/10, 7/10). That spread is your noise floor — a post-sabotage drop needs to be clearly bigger than that to count as real signal, not an artifact of variance you'd have seen anyway.
- **Lower temperature** on both the pipeline call and the judge call, where your project allows it — this shrinks (though doesn't eliminate) the noise floor you're trying to see past.
- **Per-task comparison, not just overall score** — a specific task flipping from PASS to FAIL, tied to the exact instruction you removed, is much stronger evidence than "the overall number went down by one," which could have several explanations even after averaging multiple runs.
- **A minimum-drop threshold**, decided *before* you look at the sabotaged result — e.g. "I'll only call this a successful catch if at least 2 specific tasks that were PASS are now FAIL, not just a 1-point overall wobble." Deciding this after seeing the result lets you unconsciously rationalize a borderline number either way.

Before running the sabotage, decide your minimum-drop threshold and write it down — then check Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both describe a single before/after comparison — accurate for a fully deterministic suite, but incomplete the moment any part of the suite touches an LLM. Advanced adds the missing piece: establishing a noise floor from repeated baseline runs, and deciding a real drop threshold in advance, so a genuine regression catch can be told apart from ordinary LLM run-to-run variance instead of being asserted on a single pair of numbers.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
step 1: run the suite against the current, working prompt
        write down: score_before

step 2: back up the prompt file

step 3: edit the prompt, remove one real instruction
        (example: remove "always mention the return policy")

step 4: run the exact same suite again
        write down: score_after

step 5: compare
        if score_after is clearly lower than score_before: good, the suite works
        if score_after is about the same: the suite has a hole, go fix it

step 6: restore the original prompt from the backup
```

Here's almost the whole thing — just try running it and reading it line by line:

```
baseline_score = run_eval_suite()
print("baseline:", baseline_score)

# now go edit the prompt file by hand – remove one real instruction – then:
sabotaged_score = run_eval_suite()
print("after sabotage:", sabotaged_score)

assert sabotaged_score < baseline_score
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
baseline = run_eval_suite(prompt="current/working prompt")
record baseline.score, and which individual tasks passed/failed

backup the current prompt file

sabotaged_prompt = current prompt, with one real instruction deleted

sabotaged = run_eval_suite(prompt=sabotaged_prompt)
record sabotaged.score, and which individual tasks passed/failed

compare:
    if sabotaged.score < baseline.score by a meaningful amount: suite works as in
    if sabotaged.score ~= baseline.score: find which specific task SHOULD have ca
        and tighten its rule

restore the original prompt from the backup before moving on
```

```
baseline_result = run_eval_suite()
print(f"baseline: {baseline_result.overall_score} ({baseline_result.passed}/{base

# edit the prompt file by hand between these two calls – remove one real instruct

sabotaged_result = run_eval_suite()
print(f"after sabotage: {sabotaged_result.overall_score} ({sabotaged_result.passe

assert sabotaged_result.overall_score < baseline_result.overall_score
```

What's missing: code that identifies *which specific tasks* newly failed between the two runs, not just the overall score. Write it yourself before moving to Advanced.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
function run_n_baselines(n):
    scores = [run_eval_suite().overall_score for _ in range(n)]
    return scores # this spread IS your noise floor

baseline_scores = run_n_baselines(3)
noise_floor = max(baseline_scores) - min(baseline_scores)

backup the prompt, then sabotage it (remove one real instruction)

sabotaged = run_eval_suite()
drop = average(baseline_scores) - sabotaged.overall_score

decide BEFORE looking: minimum_drop_to_count_as_real = noise_floor + some margin

if drop > minimum_drop_to_count_as_real: suite caught a real regression
else: drop might just be noise – look at per-task results instead of trusting the

restore the original prompt from the backup
```

Here's almost the whole thing — fill in the decision logic yourself:

```
def run_n_baselines(n: int = 3) -> list[float]:
    return [run_eval_suite().overall_score for _ in range(n)]

baseline_scores = run_n_baselines(3)
noise_floor = max(baseline_scores) - min(baseline_scores)
print(f"baseline scores: {baseline_scores}, noise floor: {noise_floor:.2f}")

# now edit the prompt file by hand – remove one real instruction – then:
sabotaged_result = run_eval_suite()
average_baseline = sum(baseline_scores) / len(baseline_scores)
drop = average_baseline - sabotaged_result.overall_score

# your turn: decide a real threshold using noise_floor, and compare drop against
# don't just check `drop > 0`, that doesn't account for what you just measured as
...
```

Fill in the threshold decision and per-task diff logic yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both compare exactly 2 numbers — one before, one after — and trust the difference. Advanced runs the baseline 3 times first to measure how much the score naturally wobbles with *no* prompt change at all, then requires the post-sabotage drop to clearly exceed that wobble before calling it a real catch — turning "the score went down" into "the score went down by more than it ever does on its own," which is the actual claim this exercise is asking you to prove.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)